

EECS 489 - WN 25

Discussion 9

ARP, Project 4, Wireshark, Modern C++

Announcements

Assignment 4 has been released! Due on **Tuesday, April 22.**

You have 3 group late days in total for Assignment 2-4.

Agenda

- Exercises: Address Resolution Protocol (ARP)
- Project 4 Introduction
- Wireshark
- Modern C++

Address Resolution Protocol (ARP)

Address Resolution Protocol

- When communicating over local area network (LAN), you send messages using **ethernet frames**, which locates a device using its MAC address.
- If you know a device's IP address, but not its MAC address, you must use **ARP** to figure it out.
 - Network devices can broadcast ARP Requests: *What is the MAC address of the device with IP address xyz?*
 - Sent to broadcast MAC address FF:FF:FF:FF:FF:FF
 - Includes its own IP, MAC address for response
 - The device with IP address xyz will send an ARP Response with its MAC address
 - This will be sent to the specific MAC address of the sender, not broadcasted

Address Resolution Protocol

- To make resolution faster, hosts typically use an **ARP Cache**
 - Maintains some sort of IP → MAC mapping
 - Like other things in networking, entries expire after some TTL

Address Resolution Protocol

- To make resolution faster, hosts typically use an **ARP Cache**
 - Maintains some sort of IP → MAC mapping
 - Like other things in networking, entries expire after some TTL

Address Resolution Protocol

- To make resolution faster, hosts typically use an **ARP Cache**
 - Maintains some sort of IP $\rightarrow$ MAC mapping
 - Like other things in networking, entries expire after some TTL

Internet Protocol (IPv4) over Ethernet ARP packet

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Hardware Type (1)															Protocol Type (0x0800)																
4	32	Hardware Length (6)								Protocol Length (4)								Operation															
8	64	Sender Hardware Address																															
12	96																																Sender Protocol Address
16	128	↶	Sender Protocol Address (cont.)														Target Hardware Address																
20	160																																
24	192	Target Protocol Address																															

Exercise 1

A computer with IP address 192.168.1.2 wants to communicate with another computer on the same local network with IP address 192.168.1.5. What ARP messages would be sent in this case?

Exercise 1

A computer with IP address 192.168.1.2 wants to communicate with another computer on the same local network with IP address 192.168.1.5. What ARP messages would be sent in this case?

- The computer with IP address 192.168.1.2 will send an ARP request asking, “Who has IP address 192.168.1.5?” This request is broadcast to all devices on the local network.
- The computer with IP address 192.168.1.5 will respond with an ARP reply containing its MAC address. The requesting computer then stores this information in its ARP cache.

Exercise 2

Suppose a host with IP address 10.0.0.2/24 wants to communicate with a device at 10.0.1.3.

Would the first host send an ARP request in this case?

Exercise 2

Suppose a host with IP address 10.0.0.2/24 wants to communicate with a device at 10.0.1.3. Would the first host send an ARP request in this case?

ARP is used to resolve MAC addresses for devices within the same local network. Since 10.0.0.2/24 and 10.0.1.3 are on different subnets, the host with 10.0.0.2 will send the packet to its configured gateway (router) rather than using ARP. The router will handle forwarding the packet to the destination subnet.

Exercise 3

Host A has an ARP entry for Host B's IP address, with a timeout of 10 minutes. If Host B changes its MAC address (e.g., due to network adapter replacement) within this time, explain what will happen the next time Host A tries to reach Host B.

Exercise 3

Host A has an ARP entry for Host B's IP address, with a timeout of 10 minutes. If Host B changes its MAC address (e.g., due to network adapter replacement) within this time, explain what will happen the next time Host A tries to reach Host B.

If Host B's MAC address has changed, Host A's ARP entry for Host B will become stale. However, as long as the entry has not expired, Host A will attempt to use the outdated MAC address, causing communication failure. Once the entry times out, Host A will send a new ARP request and get the updated MAC address of Host B, restoring communication.

Exercise 4

What security vulnerabilities can you think of with ARP?

Exercise 4

What security vulnerabilities can you think of with ARP?

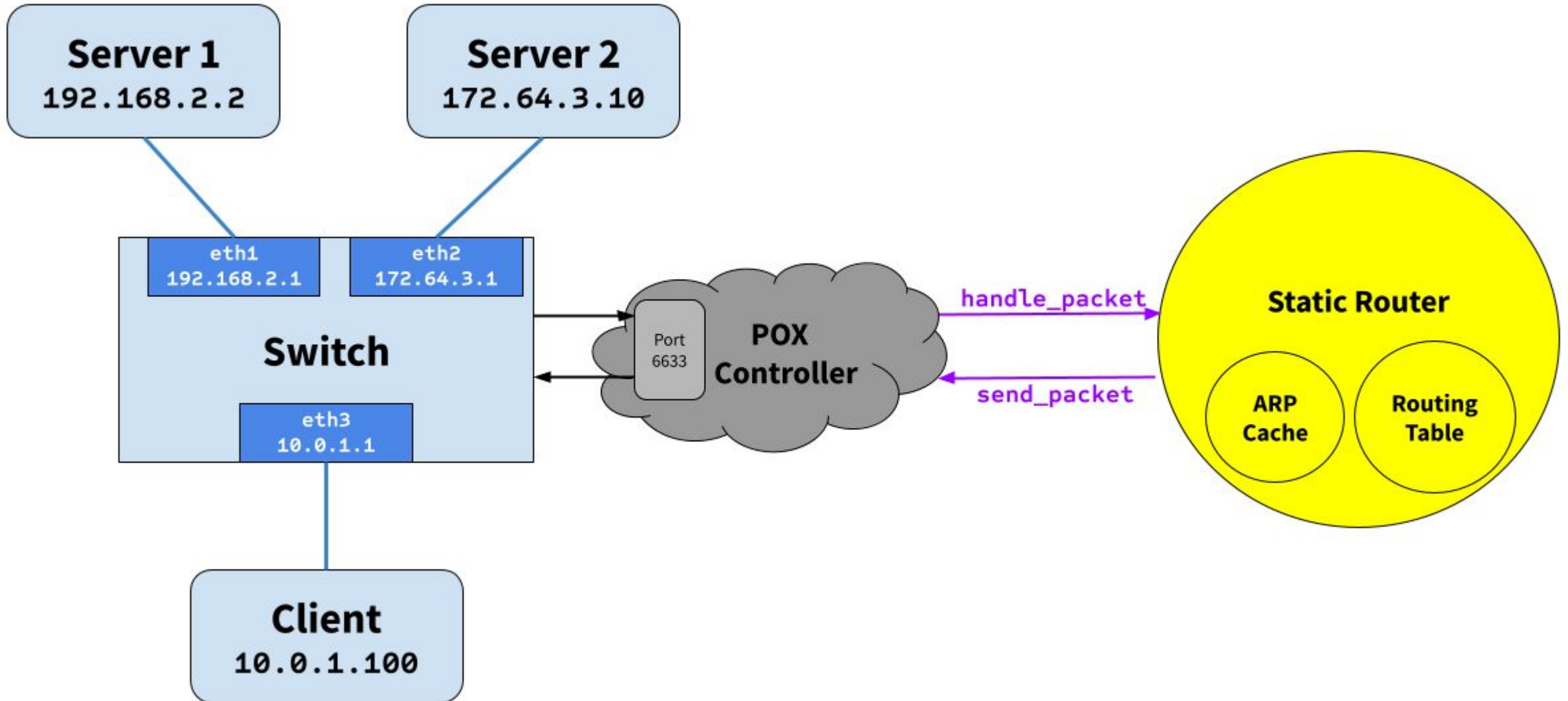
A man-in-the-middle attack can occur using **ARP spoofing**.

- The attacker can send falsified ARP responses to Host A, claiming that the attacker's MAC address is associated with Host B's IP address.
- Similarly, the attacker can send ARP responses to Host B, claiming the same MAC address is associated with Host A's IP address.
- This causes both hosts to send their packets to the attacker, who can then intercept or modify the traffic.

Prevention: Static MAC Addresses (when possible), Monitoring Software

Project 4

Project 4 Architecture



Project 4 Capabilities

Once your router is functioning correctly, you will be able to perform all of the following operations:

- Ping any of the switch's interfaces from the hosts.
- Traceroute to any of the switch's interface IP addresses.
- Ping any of the hosts from any other host.
- Traceroute to any of the hosts from any other host.
- Download a file using HTTP from one of the HTTP servers.

Setup

Run everything on your VM from Project 1.

- Clone the repo and run setup instructions
- Code locally and push to a private repo or SSH into your VM to code
- Run in your VM:

```
./run_pox.sh
```

```
./run_mininet.sh
```

```
./StaticRouter -r rtable
```

```
mininet> client ping server1
```

Implementation Part 1: RoutingTable

- Some basic functionality for reading the routing table and implementing a longest prefix match algorithm.
- Our routing table is very simple – make sure you are able to handle more complicated tables! Write your own test cases :)

prefix	next_hop	netmask	interface
0.0.0.0	10.0.1.100	0.0.0.0	eth3
192.168.2.2	192.168.2.2	255.255.255.255	eth1
172.64.3.10	172.64.3.10	255.255.255.255	eth2

Implementation Part 2: ARPCache

- tick() is called every 100ms – you should delete expired entries from the cache
- addEntry() should be called by your router when it gets an ARP response
- getEntry() should be called by your router when it wants to know the MAC address of a given IP address
- queuePacket() is called when a packet needs to wait on an ARP request
- **You will be responsible for implementing all of these!**

```
class ArpCache : public IArpCache {
public:
    ArpCache(std::chrono::milliseconds timeout,
             std::shared_ptr<IPacketSender> packetSender, std::shared_ptr<IRoutingTable> routingTable) : timeout(timeout), packetSender(packetSender), routingTable(routingTable) {}

    ~ArpCache() override;

    void tick();

    void addEntry(uint32_t ip, const mac_addr& mac) override;

    std::optional<mac_addr> getEntry(uint32_t ip) override;

    void queuePacket(uint32_t ip, const Packet& packet, const std::string& iface) override;

private:
    void loop();
};
```

Implementation Part 3: Static Router

```
class StaticRouter {
public:
    StaticRouter(std::unique_ptr<IArpCache> arpCache, std::shared_ptr<IRoutingTable> routingTable,
                std::shared_ptr<IPacketSender> packetSender);

    /**
     * @brief Handles an incoming packet, telling the switch to send out the necessary packets.
     * @param packet The incoming packet.
     * @param iface The interface on which the packet was received.
     */
    void handlePacket(std::vector<uint8_t> packet, std::string iface);

private:
    std::mutex mutex;

    std::shared_ptr<IRoutingTable> routingTable;
    std::shared_ptr<IPacketSender> packetSender;

    std::unique_ptr<IArpCache> arpCache;
};
```

Implement this!

Use these!

ICMP Protocol

Internet Control Message Protocol – used to communicate error messages and other control information between hosts using IP.

- ICMP header will live below IP header, much like a TCP/UDP header.
- ICMP is **not** a L4 protocol – it is associated with IP.

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Type								Code								Checksum															
4	32	Rest of Header																															

ICMP Protocol

Type	Code	Description
0 – Echo Reply	0	Echo reply
3 – Destination Unreachable	0	Destination network unreachable
	1	Destination host unreachable
	2	Destination protocol unreachable
	3	Destination port unreachable
	4	Fragmentation needed and DF flag set
	5	Source route failed
5 – Redirect Message	0	Redirect datagram for the Network
	1	Redirect datagram for the host
	2	Redirect datagram for the Type of Service and Network
	3	Redirect datagram for the Service and Host

8 – Echo Request	0	Echo request
9 – Router Advertisement	0	Use to discover the addresses of operational routers
10 – Router Solicitation	0	
11 – Time Exceeded	0	Time to live exceeded in transit
	1	Fragment reassembly time exceeded
12 – Parameter Problem	0	Pointer indicates error
	1	Missing required option
	2	Bad length
13 – Timestamp	0	Used for time synchronization
14 – Timestamp Reply	0	Reply to Timestamp message

- ping:
 - Type 8, Code 0 (request)
 - Type 0, Code 0 (reply)
- Packet cannot be forwarded
 - Reply with Type 3, Code 0/1

ICMP Protocol

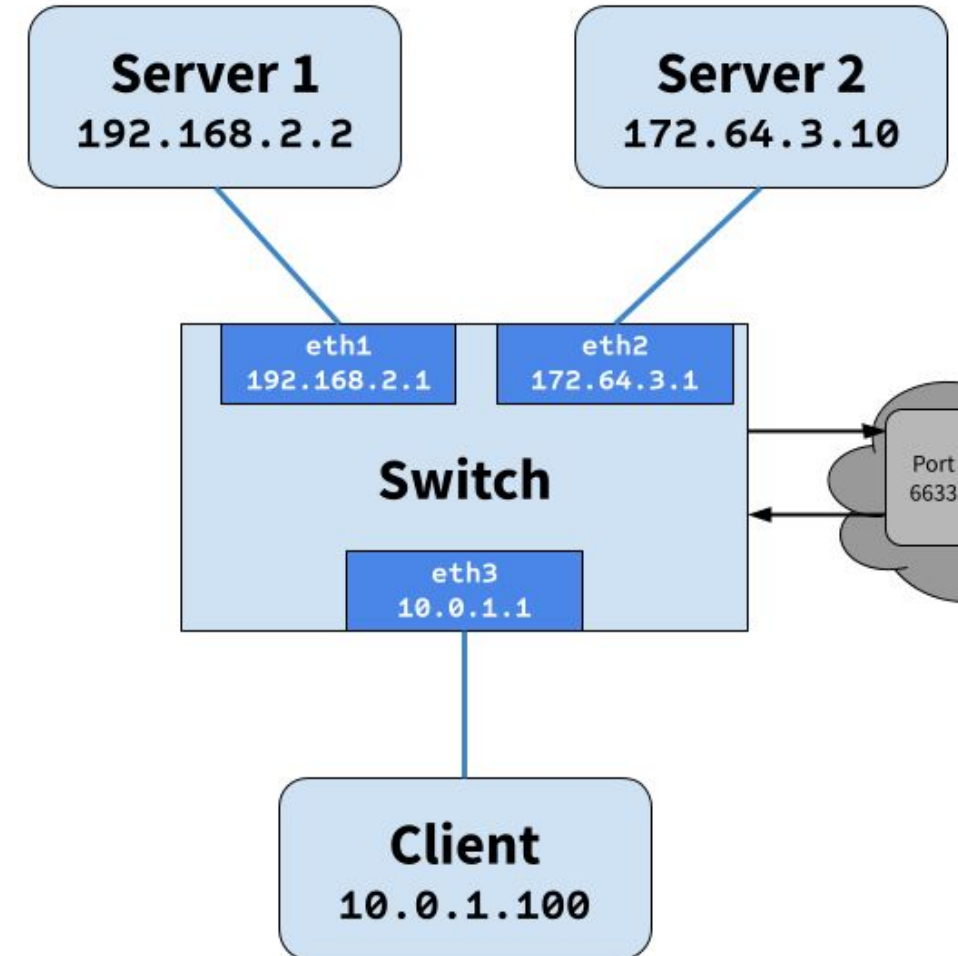
ICMP messages you will need to generate:

- **Echo reply (type 0):** Sent in response to an echo request (`ping`) to one of the router's interfaces. (This is only for echo requests to any of the router's IPs. An echo request sent elsewhere should be forwarded).
- **Destination net unreachable (type 3, code 0):** Sent if there is a non-existent route to the destination IP (no matching entry in routing table when forwarding an IP packet).
- **Destination host unreachable (type 3, code 1):** Sent after seven ARP requests were sent to the next-hop IP without a response.
- **Port unreachable (type 3, code 3):** Sent if an IP packet containing a UDP or TCP payload is sent to one of the router's interfaces. This is needed for `traceroute` to work.
- **Time exceeded (type 11, code 0):** Sent if an IP packet is discarded during processing because the TTL field is 0. This is also needed for `traceroute` to work.

PLEASE use a helper function (we are begging)

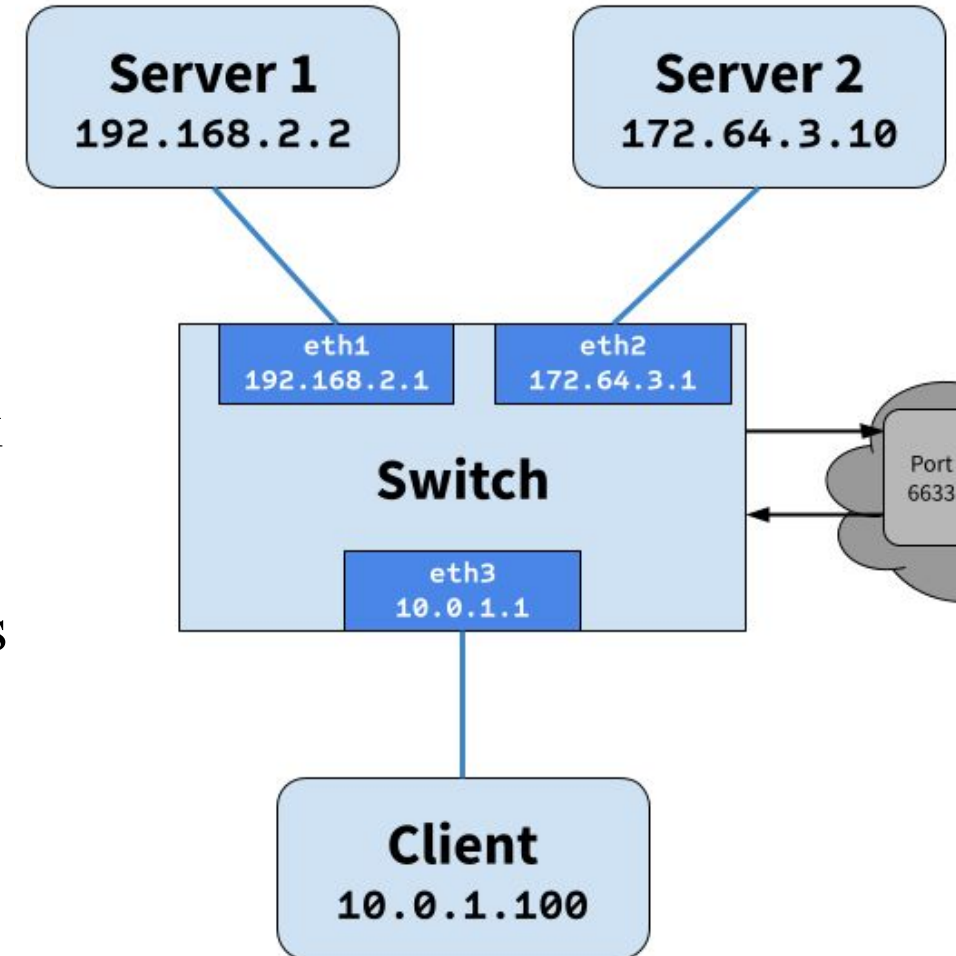
Common Confusion: IP Addresses

- You may notice that every **interface** on the mininet switch has its own IP address — what's going on with that?
- IP addresses are not associated with machines (although we are often sloppy and think of them that way) – they are associated with individual network interfaces.
- L3 Routers have multiple interfaces, each of which might connect to a different LAN (in this case, we simplify and just connect to different hosts).



Common Confusion: IP Addresses

- IP addresses are not associated with machines – they are associated with individual network interfaces.
- L3 Routers have multiple interfaces, each of which might connect to a different LAN.
- Each interface has an address within the subnet mask of the network it connects to
 - Appears to live in the local network and serves as a default gateway.



Debugging Project 4

How can we debug the router?

- Use a debugger – see where the router gets stuck when `handle_packet` is called
 - print debugging is also very helpful
- Try pinging a router interface from Mininet
- Try traceroute from one Mininet host to another
- Try doing `wget index.html` from one of the servers from the client in Mininet
- **Use packet captures to understand what is happening under the hood**

How can we debug the router?

Protocols: Logging Packets

You can log the packets received and generated by your SR program by using the `-l` parameter. The file will be in `pcap` format, so you can use [Wireshark](#) or `tcpdump` to read it. Wireshark is free to download, and a powerful tool for a variety of applications -- we encourage you to use this project to become familiar with its basic functionality.

For example,

```
./StaticRouter -p my_prefix
```



will output `my_prefix_input.pcap` and `my_prefix_output.pcap`.

How can we debug the router?

- `...input.pcap`: Any packets arriving at the router
- `...output.pcap`: Any packets being sent by your router
- See demo files in Discussion folder on Google Drive

Wireshark: Ping Server from Client

- Demo (find in GitHub):

Discussion/discussion_pcaps/ping_ok_client_server_*.pcap

- Can click into packets to expand them
- Can merge two packet captures
- Can inspect individual layers

```
mininet> client ping server1
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=293 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=166 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=190 ms
64 bytes from 192.168.2.2: icmp_seq=4 ttl=63 time=106 ms
64 bytes from 192.168.2.2: icmp_seq=5 ttl=63 time=95.0 ms
64 bytes from 192.168.2.2: icmp_seq=6 ttl=63 time=163 ms
^C
--- 192.168.2.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5006ms
rtt min/avg/max/mdev = 95.036/168.832/292.819/64.872 ms
```

Wireshark: Buggy Ping

- Demo: `ping_err_client_server_*.pcap`
- Takeaway: Use Wireshark settings to your advantage

```
mininet> client ping server1
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
^C
--- 192.168.2.2 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3111ms
```

Wireshark: Correct Traceroute

- Demo: **traceroute_ok_*.pcap**
- Be sure to install traceroute on your EC2 machine!
`sudo apt-get install traceroute`
- traceroute works by sending packets with increasing TTLs so that intermediate packets respond to it with a ICMP: Time to live exceeded message

```
mininet> client traceroute server1
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 60 byte packets
 1  ip-10-0-1-1.ec2.internal (10.0.1.1)  44.526 ms  44.521 ms  44.521 ms
 2  ip-192-168-2-2.ec2.internal (192.168.2.2)  237.790 ms  237.814 ms  238.492 ms
```

Wireshark: Bad Traceroute

- Demo: `traceroute_ok_*.pcap`

```
mininet> client traceroute server1
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 60 byte packets
 1  * * *
 2  ip-192-168-2-2.ec2.internal (192.168.2.2)  246.928 ms  246.929 ms  247.109 ms
```

Other P4 tips

- Make a flowchart of what you need to branch on and what checks you need to do for any incoming packet
 - Read the spec carefully and do this before you start coding
- Helper functions!!! Use them!!!
- Use simple command-line tools like wget, ping, traceroute, etc. to check whether everything is working as intended

Project 4 C++ Features

std::optional<T>

Introduced in C++ 17, `#include <optional>`

Motivation: What if we want to return something that may not be present?

```
?? get_student_name(uint32_t student_id) {  
    auto response = my_api.query(student_id);  
    if (response.found) {  
        return std::string(response.data);  
    } else {  
        ??;  
    }  
}
```

std::optional<T>

Motivation: What if we want to return something that may not be present?

Option 1: Return a sentinel value?

```
std::string get_student_name(uint32_t student_id) {  
    auto response = my_api.query(student_id);  
    if (response.found) {  
        return std::string(response.data);  
    } else {  
        return "null";  
    }  
}
```

std::optional<T>

Motivation: What if we want to return something that may not be present?

Option 2: Throw an exception?

```
std::string get_student_name(uint32_t student_id) {  
    auto response = my_api.query(student_id);  
    if (response.found) {  
        return std::string(response.data);  
    } else {  
        throw std::runtime_error("Student not found!");  
    }  
}
```

std::optional<T>

Motivation: What if we want to return something that may not be present?

Option 3: Use pointers/iterators?

```
std::string* get_student_name(uint32_t student_id) {  
    auto response = my_api.query(student_id);  
    if (response.found) {  
        return new std::string(response.data);  
    } else {  
        return nullptr;  
    }  
}
```

std::optional<T>

Motivation: What if we want to return something that may not be present?

Problems with the previous solutions:

- Sentinel values are prone to hard-to-debug edge cases (what if comedian [Luke Null](#) signs up for our class?)
- Exceptions should be exceptional, not finding a student in a query is not
 - *Aside:* Because of performance implications, try/catch is almost never used in real C++ code. Instead, opt for std::optional like structs. For an example, see [here](#).
- Returning nullptr to indicate the lack of a value is essentially a sentinel and can lead to difficult-to-debug segfaults.

std::optional<T>

Motivation: What if we want to return something that may not be present?

What we actually want: Meaningful and typeful semantics to indicate “Some [x]” or “None”

- Ensure that the client does not need too much additional scaffolding
- Ensure that the client can handle error case gracefully

Answer: `std::optional<T>`

Aside: This is an instance of a sum type

- A type that can take on one value out of a set of options
- There is support for such types in C++ (std::variant, but requires knowledge of C++ templates)
- Take EECS 490 for more!

std::optional<T>

```
std::optional<std::string> get_student_name(uint32_t student_id) {
    auto response = my_api.query(student_id);
    if (response.found) {
        return std::string(response.data);
    } else {
        return std::nullopt;
    }
}

int main() {
    // ...
    std::optional<std::string> student_name = get_student_name(student_id);
    if (!student_name) {
        std::cout << "Student not found!\n";
        return 1;
    }
    std::cout << "Student with ID's name is " << *student_name
        << " with length" << student_name->size() << '\n';
    return 0;
}
```

`std::optional<T>`

- `opt.has_value()`: checks if the object represents a valid state
- `opt.value()`: returns a reference to the underlying value, assume state is valid
 - Throws `std::bad_optional_access` if the state is invalid (i.e. the optional is empty)
- `opt.value_or(default)`: returns a reference to the underlying value, or returns default if no value is present
- `opt.reset()`: set the state to invalid
- `operator*`, `operator->`: access underlying object
- `std::make_optional(args...)`: constructs a valid object in the wrapper

std::optional<T> in P4

```
class RoutingTable : public IRoutingTable {
public:
    /**
     * @brief Constructs a RoutingTable object from a given file path.
     * @param routingTablePath The path to the file containing routing table entries.
     */
    explicit RoutingTable(const std::filesystem::path& routingTablePath);

    std::optional<RoutingEntry> getRoutingEntry(ip_addr ip) override;
};
```

Also in ARPCache:

```
std::optional<mac_addr> getEntry(uint32_t ip) override;
```

Smart pointers

Motivation: Managing dynamic memory is hard and error-prone :(

[Chromium](#) > [Chromium Security](#) >

Memory safety

The Chromium project finds that around 70% of our serious security bugs are [memory safety problems](#). Our next major project is to prevent such bugs at source.

The problem

Around 70% of our high severity security bugs are memory unsafety problems (that is, mistakes with C/C++ pointers). Half of those are use-after-free bugs.

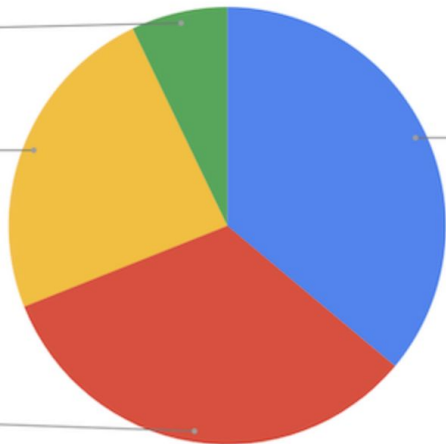
High+, impacting stable

Security-related assert
7.1%

Other
23.9%

Use-after-free
36.1%

Other memory unsafety
32.9%



Administration

FEBRUARY 26, 2024

Statements of Support for Software Measurability and Memory Safety



ONCD > BRIEFING ROOM > PRESS RELEASE

White House urges developers to dump C and C++

News

Feb 27, 2024 • 5 mins

Application Security

C Language

Programming Languages

Biden administration calls for developers to embrace memory-safe programming languages and move away from those that cause buffer overflows and other memory access vulnerabilities.

Related content

Smart pointers

What are smart pointers?

- Class objects that behave like raw pointers
- Acquires dynamic memory upon initialization
- **Automatically** de-allocates memory upon destruction
- Example of **RAII** – Resource Allocation Is Initialization

Smart pointers

This is the general idea (not a real implementation) – when a smart pointer goes out of scope, allocated memory is deleted!

```
1  template<typename T>
2  class SmartPtr {
3  public:
4      explicit SmartPtr(T *p_in) : ptr{ p_in } {}
5
6      ~SmartPtr() { delete ptr; }
7
8      T &operator*() { return *ptr; }
9      T *get() { return ptr; }
10
11 private:
12     T *ptr;
13 };
```

Smart pointers

Why smart pointers?

- You will rarely have to worry about memory leaks (more on this later)
- Make sure their lifetimes last exactly as long as you need them to
- Be careful if they go out of scope too early, or they never go out of scope

Smart pointers

`unique_ptr<T>`

- Implements **unique ownership**
- 1 and only 1 `unique_ptr` owns an object. When the only `unique_ptr` pointing to an object is destroyed, the underlying object is **automatically destroyed**.
- Preferred over `shared_ptr` (when something should only be in 1 place)

Smart pointers

`shared_ptr<T>`

- Implements **shared ownership**
- Any number of `shared_ptr`s can jointly own an object. When the last `shared_ptr` pointing to an object dies, the underlying object is destroyed.

`weak_ptr<T>`

- Implements **non-owning** (weak) reference to an object owned by `shared_ptr`s
- Can obtain a `shared_ptr` from it with `.lock()`
- Useful for tracking whether an object managed by `shared_ptr` is alive

unique_ptr<T>

- Implements **unique ownership**
 - **ONE** unique_ptr to a given object at a time
- How is that enforced?

```
// delete copy constructor, assignment operator
unique_ptr(unique_ptr const&) = delete;
unique_ptr& operator=(unique_ptr const&) = delete;
```

- Accidental copying of a unique_ptr will be caught at *compile time*
 - Principle of “failing early”

unique_ptr<T>

- `std::make_unique<T>()`
 - Safe way to construct a `unique_ptr` with a non-null underlying object
 - Returns a newly constructed `unique_ptr`
- `reset()`
 - Resets the `unique_ptr` to represent a `nullptr`
 - Delete underlying object if exists
- `release()`
 - Resets the `unique_ptr` to represent a `nullptr`
 - Does NOT delete the underlying object, return a raw pointer to it instead
- `operator*`, `operator->`, `operator bool`
 - Access the smart pointer like a raw pointer
 - Pass the smart pointer as a boolean like a raw pointer

unique_ptr<T>

How to transfer ownership of a unique_ptr? **Move semantics!**

```
StaticRouter::StaticRouter(std::unique_ptr<IArpCache> arpCache, std::shared_ptr<IRoutingTable> routingTable,  
                           std::shared_ptr<IPacketSender> packetSender)  
    : routingTable(routingTable)  
      , packetSender(packetSender)  
      , arpCache(std::move(arpCache))  
{  
}
```

shared_ptr<T>

- Implements **reference-counted** shared ownership
- What is a reference count?
 - An integer that tracks how many instances of the shared_ptr object manages the address it points to
 - Increments upon value or copy construction
 - Decrements upon destruction
 - Does **NOT** change upon move construction
- Automatically delete the managed pointer when ref count == 0
- *Used in conjunction with weak_ptr to prevent circular issues – read about this on your own!*

Design Decisions

```
StaticRouter::StaticRouter(std::unique_ptr<IArpCache> arpCache, std::shared_ptr<IRoutingTable> routingTable,  
                           std::shared_ptr<IPacketSender> packetSender)  
    : routingTable(routingTable)  
      , packetSender(packetSender)  
      , arpCache(std::move(arpCache))  
{  
}
```

The StaticRouter should be the **only** owner of its arpCache, so we use a unique_ptr here.

In theory, multiple things could use the packetSender or the routingTable, so this is a shared_ptr (though in this project, we could have used unique_ptr here as well).

Design Decisions

Use a `unique_ptr` when....

- By default – unique ownership is stronger than shared ownership!
- E.g. have a data structure of `unique_ptr`s to database connection
 - This is where these objects “live” – nothing else should be able to create/destroy them. You can pass around raw pointers to functions to work with these objects.

Use a `shared_ptr` when...

- Multiple functions, threads, or components need access to an object at the same time, and it isn't clear how they overlap/interact with each other.
- E.g. you have one “logger” that is being used by multiple components in your system
- Note: a bit more overhead than `unique_ptr`s due to refcounts!

Caution

- Smart pointers are not foolproof! If you use them in bad ways, you will end up encountering the same segfaults and uninitialized memory bugs :(
- E.g. if you are using smart pointers, but still extract a raw pointer using `.get()` and do a `delete` – you will get a segfault!
 - Smart pointers are to save you from yourself – follow the common design patterns, and they will save you a lot of time and bugs.
 - No easy way to understand them until you use them!

Thanks!